

VIVE Improvement Plan — Simple Summary (Updated)

Simplified summary of the priority list and roadmap. Full technical detail lives in the longer architecture review. Reflects the reviewer dashboard decision (Streamlit), the AI extraction decision (Claude), and a confirmed multi-user requirement (5-10 people, mixed often/occasional use).

Priority Table

Features	What it means (simple)	Priority	Complexity	Impact
Docker	Packages the whole project so it runs the same on any computer, no manual setup.	High	Low	High
Rulebook	Writes down every “don’t undo this” decision so nobody breaks something on purpose-built.	High	Low	High
Disposition model	Remembers past decisions on exceptions, so the same issue isn’t reviewed every month.	High	Low-Med	High
Real shared database	Confirmed need: 5-10 people will use this, often and occasionally. Replaces the single file so several people can use it at once without errors.	High	Med-High	High
Job queue	So uploads don’t block each other — with several people using it, one person’s upload processes in the background instead of freezing the page for everyone else.	High	Medium	High
Per-user logins	Each person gets their own login. Needed so the disposition record actually shows who resolved what, once more than one person is using it.	High	Low-Med	High
Shared hosting	The dashboard needs to run somewhere everyone can reach, all the time — not on one person’s laptop anymore.	High	Low	High
Object storage (Blob)	Keeps a permanent, safe copy of every original PDF in the cloud.	Medium	Low	Medium
Migration tooling	Keeps a clear record of every database change, so nobody has to guess its state.	Medium	Low	Medium
Reviewer dashboard	A complete Streamlit page — upload a vendor PDF and it runs the pipeline directly (not just a viewer), plus results/history and a way to resolve exceptions. Now sits on top of the real database, job queue, and per-user logins above, since several people will use it.	Medium	Low-Med	Medium-High
Confidence gate	Pauses the whole document (not just one row) if AI isn’t sure about any part of it.	Medium	Medium	Medium
Dependency-skip check	If one field is missing, clearly skip just that row instead of guessing or crashing.	Medium	Low-Med	Medium
Audit log	Keeps a record of who resolved which exception, and when.	Medium	Low	Medium
Retry/truncation fix	Notifies when the AI’s answer got cut off and jumps straight to backup, no wasted time.	Low	Low	Low-Med

Features	What it means (simple)	Priority	Complexity	Impact
Config cleanup	Moves a couple of hardcoded numbers into the settings file for easier tuning.	Low	Low	Low-Med

AI Model Decision (Updated)

- Claude (Haiku 4.5) is the extraction engine — already available with no new setup required, and comfortably handles VIVE's PDF volume without rate-limit issues.
- pdfplumber + OCR (Tesseract) is the last-resort fallback if Claude is unavailable: free, no AI, and — now that Docker (see above) removes setup complexity — low-maintenance to keep. Final chain: Claude → pdfplumber + OCR.
- Haiku 4.5 specifically, not Sonnet or Opus — extraction is a simple-enough task (reading numbers and text off a PDF) that a stronger reasoning tier wouldn't add value, and Haiku keeps per-document cost low at VIVE's volume.
- Claude is also still used for the optional --explain narrative step, as before.
- Which method actually processed a document (Claude vs. the pdfplumber+OCR fallback) is already logged — document_intake_log.extraction_method and ai_audit_log.ai_provider already exist in VIVE's schema. No new tracking needs to be built.
- The reviewer dashboard still uses Streamlit — that choice doesn't change with confirmed multi-user demand, since Streamlit comfortably handles 5-10 concurrent users. What changes is what runs behind it: a small background worker now processes the upload queue, and a real shared database replaces the single file. Still all Python — not a jump to Scrutin's full separate frontend + backend stack.

How Storage & the Job Queue Actually Work (New)

- Each PDF is saved in Blob Storage under a path like vendor-name/year/month/document-hash.pdf — using the same SHA-256 hash VIVE already computes for caching, so the same file is never stored twice.
- Alongside each file: who uploaded it, when, and which vendor — saved as metadata, not just guessed from the filename.
- A new column on the existing document log links each processed statement straight to its saved file, so “pull up the original PDF for this exception” is always possible later.
- The job queue is a simple table: each upload gets a row (queued → processing → done/failed), and a small always-running worker script picks up one job at a time and runs it.
- If a worker crashes mid-job, that job doesn't get stuck forever — it's automatically re-queued after sitting too long with no progress.
- The same document is never accidentally processed twice, even if it somehow gets queued twice — reuses the existing cache check.

New Costs to Confirm

- Object storage (Blob): a few cents a month at VIVE's volume — negligible.
- Real shared database (Azure SQL, Basic tier): roughly \$5/month — small, but new; didn't exist before.
- Shared hosting for the dashboard (Azure App Service, Basic tier): roughly \$13/month.
- Total new recurring cost, rough estimate: about \$18-20/month combined — the actual number to bring for sign-off, not “a few dollars.”

Not Suitable for VIVE Right Now

Item	Why not
AI backup providers, salvage, caching	VIVE already does these better than Scrutin — nothing to change here.
Full role-based permissions (e.g. Admin vs. Reviewer)	Everyone using the dashboard is doing the same job today — per-user logins cover the current need without needing separate permission levels yet.

Roadmap — What Order to Build Things In

Phase	Features included	Timeframe	Why this order
Phase 1	Docker, Rulebook, Migration tooling	1-2 weeks	Cheap, independent, fixes setup pain already felt.
Phase 2	Disposition model, Audit log, Object storage (Blob)	2-4 weeks	Biggest time-saving win, plus safe document storage.
Phase 3	Real shared database, Job queue, Per-user logins, Shared hosting, Reviewer dashboard	3-6 weeks	Confirmed multi-user need (5-10 people) — the dashboard now depends on all four of these to work properly at that scale.
Phase 4	Dependency-skip check, Retry/truncation fix, Config cleanup	As time allows	Smaller reliability polish, not urgent.
Phase 5 (only if triggered)	Confidence gate	Only if triggered	Build this only once a real need shows up.